

5 Datový model (specializace DW)

V informačním systému nemocnice je použit následující logický relační datový model, kde podtržením jsou vyznačeny klíče a italikou cizí klíče:

- Diagnóza(**Kód**, Název)
 - Lékař(**RČ**, Jméno, Příjmení, Město, Ulice, Číslo, PSČ, Odbornost, RokyPraxe)
 - Pacient(**RČ**, Jméno, Příjmení, Město, Ulice, Číslo, PSČ, *PraktickýLékařRČ*), $\text{PraktickýLékařRČ} \subseteq \text{Lékař[RČ]}$
 - LéčíSe(**RČ**, *Kód*), $\text{RČ} \subseteq \text{Pacient[RČ]}$, $\text{Kód} \subseteq \text{Diagnóza[Kód]}$
1. Znázorněte výše uvedený logický relační datový model pomocí diagramu ve zvoleném konceptuálním jazyce (ER, UML).
 2. Pokud je to potřeba, rozšiřte konceptuální model a odpovídající logický relační datový model tak, aby:
 - každý pacient se mohl léčit na libovolný počet diagnóz,
 - každý pacient měl právě jednoho praktického a mohl mít libovolný počet dalších lékařů,
 - každý pacient mohl podstoupit libovolný počet vyšetření v rámci nějaké diagnózy a naopak,
 - pacient, který je zároveň lékařem, neměl osobní údaje evidovány redundantně.

Nástin řešení

1. Tři entity (třídy): Diagnóza, Lékař, Pacient. Jeden povinný 1:N vztah mezi (praktickým) lékařem a pacienty. Jeden nepovinný 1:N vztah mezi diagnózou a pacienty.
2.
 - Vztahová tabulka LéčíSe už existuje, ale podporuje omezenější kardinalitu 1:N. Stačí změnit definici klíče. LéčíSe(*RČ*, *Kód*), *RČ* $\subseteq$ Pacient[*RČ*], *Kód* $\subseteq$ Diagnóza[*Kód*]
 - Už má právě jednoho praktického lékaře. Nutno přidat M:N vztah a odpovídající vztahovou relaci pro M:N vztah mezi lékaři a pacienty. Léčí(*PacientRČ*, *LékařRČ*), PacientRČ $\subseteq$ Pacient[*RČ*], LékařRČ $\subseteq$ Lékař[*RČ*].
 - M:N vztah Vyšetření mezi pacientem a diagnózou, a k ní odpovídající vztahová relace Vyšetření(*PacientRČ*, *Kód*), PacientRČ $\subseteq$ Pacient[*RČ*], *Kód* $\subseteq$ Diagnóza[*Kód*].
 - Nejjednodušeji tak, že třída Lékař bude podtřídou třídy Pacient, nebo obě budou podtřídou nové třídy Osoba. V prvním případě relace Lékař přijde o atributy Jméno ... PSČ a RČ bude cizím klíčem na Pacient[*RČ*]. Ve druhém bude nová relace Osoba obsahovat atributy RČ ... PSČ a obě relace Pacient a Lékař přijdou o atributy Jméno ... PSČ a RČ bude cizím klíčem na Osoba[*RČ*].

6 Transakce (specializace DW)

Pro následující transakční rozvrh, kde R znamená operaci čtení a W operaci zápisu:

	T_1	T_2	T_3
1	W(<i>Pacient</i> ₁)		
2		???	
3		W(<i>Pacient</i> ₁)	
4		COMMIT	
5			R(<i>Pacient</i> ₁)
6			W(<i>Pacient</i> ₁)
7			COMMIT
8	R(<i>Pacient</i> ₂)		
9	COMMIT		

1. Jaká databázová operace čtení/zápisu v transakci T_2 v čase 2 místo ??? způsobí, že rozvrh nebude konfliktově uspořádatelný (conflict-serializable)? Svě rozhodnutí zdůvodněte.
2. Jaká databázová operace čtení/zápisu v transakci T_2 v čase 2 místo ??? způsobí, že rozvrh nebude zotavitelný (recoverable)? Svě rozhodnutí zdůvodněte.

Nástin řešení

1. W(*Pacient*₂). Vznikne cyklus v precedenčním grafu $T_1 \rightarrow T_2$ a $T_2 \rightarrow T_1$.
2. R(*Pacient*₁). Nepotvrzené čtení hodnoty zapsané transakcí T_1 , ale COMMIT předchází konec transakce T_1 .

7 API a skriptování (specializace DW)

1. Navrhněte a popište REST API pro nemocniční systém z dřívější otázky. API musí podporovat následující funkcionalitu:
 - získání pacientů, jejichž jméno odpovídá query patternu,
 - získání informací o konkrétním pacientovi,
 - vytvořit, aktualizovat a zrušit pacienta,
 - získat seznam diagnóz pacienta,
 - přidat a odebrat diagnózu pacienta,
 - získat seznam lékařů pacienta,
 - získat seznam pacientů, kteří mají aspoň jednu ze zadaných diagnóz (seznam diagnóz může být hodně dlouhý).

2. Mějme webovou aplikaci, která zajišťuje přístup k IS přes navržené API. Napište JavaScript fragment, který po zadání query patternu (první endpoint) a kliknutí na tlačítko vyvolá příslušný HTTP API request, získá všechny pacienty odpovídající dotazu a zobrazí je v seznamu. Když uživatel klikne na konkrétního pacienta ze seznamu získaného prvním dotazem, tak se aktualizuje seznam jeho doktorů a diagnóz získáním dat z API a jejich následným zobrazením.

Předpokládejte, že API vrací data v JSON formátu a tento krátce popište formou příkladu pro každý použitý z endpointů. Předpokládejte existenci funkcí `displayDoctors` a `displayDiagnoses` pro zajištění úpravy DOM stromu. Stejně tak můžete použít funkce `clearDoctors` a `clearDiagnoses`. Dále předpokládejte existenci všech potřebných HTML elementů.

Dbejte na správné využití asynchronního zpracování požadavků. Zajistěte, aby i při nepříznivém souběhu asynchronních volání po načtení seznamu pacientů byl seznam doktorů a diagnóz v konzistentním stavu, tj. zobrazují se seznamy ke zvolenému pacientu a nezobrazují se seznamy v případě, kdy pacient vybrán nebyl. Není třeba řešit chybové stavy fetch operací ani autentikaci.

Nástin řešení

1. Příslušné endpointy by měly vypadat např. následujícím způsobem:

- (a) GET `api/v1/patients?query={pattern}`, případně GET `api/v1/patients/{pattern}`
- (b) GET/DELETE/POST `api/v1/patient/{rc}`
- (c) GET `api/v1/patient/{rc}/diagnoses`
- (d) GET/POST/DELETE `api/v1/patient/{rc}/diagnoses`
- (e) GET `api/v1/patient/{rc}/doctors`
- (f) POST `api/v1/diagnoses`

Vzhledem k tomu, že seznam diagnóz v posledním API endpointu může být dlouhý, je třeba payload posílat v těle dotazu a tedy pro tento endpoint nelze použít GET request.

2. Například takto:

```
let currentQuery = '';

document.getElementById('searchButton').addEventListener('click', async () => {
  const query = document.getElementById('searchInput').value();
  currentQuery = query;

  const patientList = document.getElementById('patient-list');

  patientList.innerHTML = '';
  clearDoctors();
  clearDiagnoses();

  const patients = await (
    await fetch(`/api/v1/patients?query=${encodeURIComponent(query)}`)
  ).json();

  patients.forEach(patient => {
    const item = document.createElement('li');
    item.textContent = patient.name;
    item.dataset.rc = patient.rc;
    item.addEventListener('click', () => fetchPatientDetails(patient.rc, query));
    patientList.appendChild(item);
  });
}

async function fetchPatientDetails(rc, query) {
  const doctors = await (await fetch(`/api/v1/patient/${rc}/doctors`)).json();
```

```

const diagnoses = await (await fetch('api/v1/patient/${rc}/diagnoses')).json;
if (query !== currentQuery){
  displayDoctors(doctors);
  displayDiagnoses(diagnoses);
}
}

```

3. Např. API se seznamem pacientu bude mít tvar:

```
[{ "rc": <RČ pacienta>, "name": <jméno>, "surname": <příjmení>, ... }]
```

4. Řešení by mělo ukazovat znalost iterace, použití async/await a práci s data atributy (list items).

5. Mělo by být zajištěno, že pokud je zavolán listener searchButtonu a zrovna se čeká na výsledky z API doktorů a diagnóz, tak tyto nepřepíšu už vyčištěné elementy.

8 Získávání informací (specializace DW)

Uvažujte situaci, kdy je třeba obohatit systém z předchozích otázek tak, aby umožňoval stanovit diagnózu na základě zadaných symptomů. Mějme kolekci dokumentů, kde každý dokument popisuje konkrétní nemoc.

1. Jak by vypadal booleovský model pro danou úlohu? Jaké termy by obsahoval specializovaný slovník pro danou úlohu? Jak jsou reprezentovány dotazy a dokumenty v booleovském modelu a jak lze dotazování v tomto modelu efektivně implementovat?
2. Řekněme, že chceme systém modifikovat tak, aby dotaz nebyl seznam symptomů, ale přímo zpráva lékařského vyšetření. Jakou modifikaci booleovského modelu je vhodné v takovém případě použít? Jak se změni reprezentace dotazu a dokumentu? Jak bude pak vyhodnocována podobnost mezi dotazem a dokumentem a proč?

Nástin řešení

1. Booleovský model
2. (a) Termy = symptomy, dokumenty = popisy nemocí
(b) Specializovaný slovník symptomů, tj. není třeba získávat slovník (jako je tomu v klasickém DISu) parsováním dokumentů
(c) Systém reprezentován binární maticí, dotazy realizovány jako bitové operace nad vektorem termů (symptomů)
(d) Efektivní reprezentace invertovaným indexem a merge sort-like zpracováním
3. Rozšíření – např. vektorový model
4. (a) Dotaz bude reprezentován stejně jako dokument
(b) Podobnost se vyhodnocuje typicky pomocí cosinové vzdálenosti v tf-idf prostoru mezi vektory dotazu a dokumentu